
EEGDUCKS: AN INTRODUCTORY PYTHON PACKAGE FOR PSYCHOLOGY STUDENTS

Shannon J. Fernandes

Psychometrician
& Visiting Faculty
Mumbai

ABSTRACT

Python packages like *MNE* and MATLAB extensions like *EEGLAB* are known throughout academia for their utility in preprocessing, analyzing, and plotting EEG data. However, the steep learning curve makes it challenging for psychology students, especially in countries like India, where such training isn't a part of the standard curriculum. The lack of familiarity with python or MATLAB syntax paired with the daunting task of navigating EEG cleaning, analyzing, and plotting may discourage students from a rather a exciting field. Thus, to deal effectively with such an issue, the library *eegducks* was created, which serves as an MNE wrapper allowing students to enter neuroimaging with little to no experience in coding to import EEG data, carry out preprocessing, plot EEG signals and the power spectrum, and also run Independent Component Analysis (ICA). The library is meant to serve as a preliminary attempt to make EEG analysis and other neuroimaging tools more accessible.

Keywords EEG · Python · MNE

1 Introduction

The standard curriculum for psychology degrees, like the Bachelor's or Master's degree in Psychology, in certain countries like India mention neuroimaging tools in a theoretical context. The student is familiarized with the basic idea of the different methods and their clinical utility. However, the practical task of working with such data or analyzing the same do not find a home in such coursework, saving a few Master's programs in Cognitive Science. The eager student is then compelled to undertake the task as self-study, navigating the different resources that are available. The lack of familiarity with the standard language used in such resources, particularly with MNE [1] or EEGLAB [2] or the mathematical or programming background needed, navigating the same becomes a challenging task.

To deal with such an issue, the python library *eegducks* was created. It's goal is to simplify the process for a novice programmer, allowing to quickly grasp the key concepts of EEG while familiaarizing themselves with the coding language. *eegducks* is designed as a training wheel to bridge the gap between the current tools and a curious student willing to undertake the task. The following sections outline the use of the library.

2 Using EEGDUCKS in Python

Since the library is intended for beginners, the current article will assume little to no background in coding with python. The reader, however, is encouraged to familiarize themselves with the basics of python through a resource of their preference or the following lecture.

Python is a friendly programming language, well suited for those picking it for the first time. To write python code one requires a compiler that can read the code and accordingly run the same. There are several options available, but if the reader would like a simpler start, they can refer to Google Colab which offers free notebooks that serve as your workspace. You can access Google Colab through any browser by heading to <https://colab.research.google.com/>. As you can see in Figure 1, the following notebook opens when you click on *New Notebook*. The orange box is the space where the code would be written and the pink box is clicked when the code block is to be run. New code blocks can be added

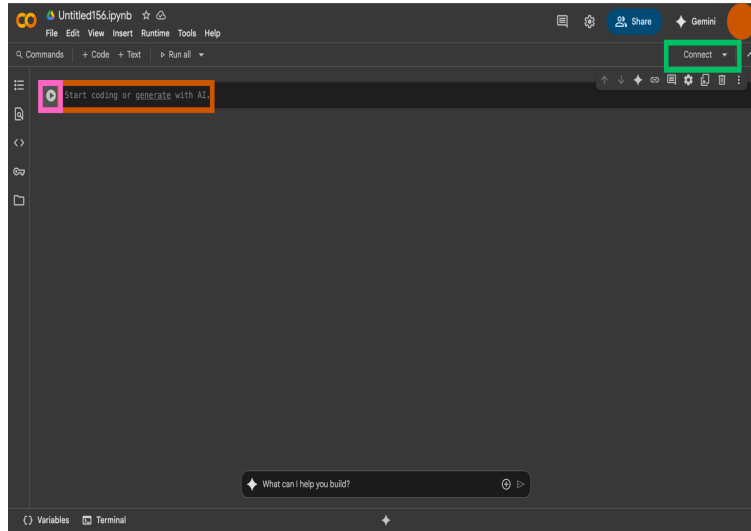


Figure 1: Google Colab Layout

by clicking on the + *Code* button. The green box shows an option *Connect* which connects to a runtime so that the code can be run. The *Connect* button can be clicked to start the runtime or clicking on the pink button to run automatically initiates the same.

To run *eegducks*, one needs to first install the library. A library can be thought of as code others have written which can be called into your work to carry out tasks quicker. To plot, for example, one could specify the layout and color and other aspects required or simply import a plotting library to do the same. To install the library, the following code needs to be run, which can also be seen in Figure 2 -

```
1 # This is a comment, it won't be run, since it's followed by the hastag symbol
2
3 # Here the exclamation mark is added before the code to install the library since
  Google Colab requires it but others don't
4
5 !pip install eegducks
```

Installing eegducks

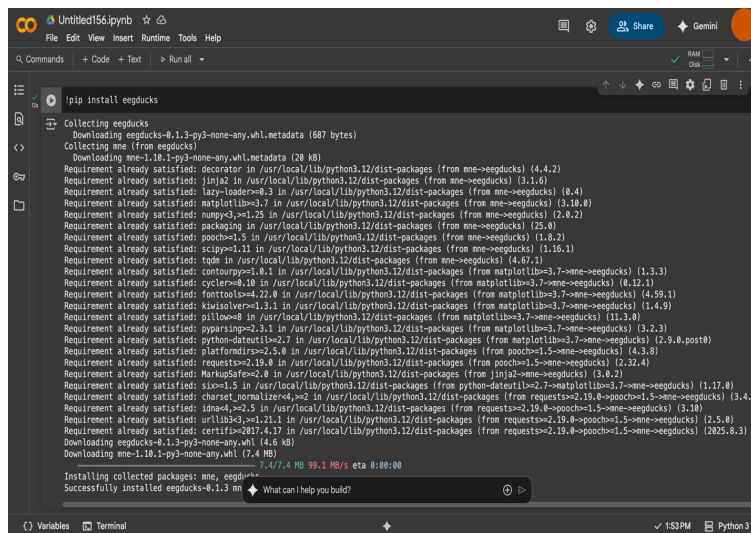


Figure 2: Google Colab Example

2.1 Loading EEG data using eegducks

Now that *eegducks* has been installed, it needs to be imported in one's code, followed by specifying the EEG files to be used. To import eegducks, you run the following code -

```
1 import eegducks as eds
```

Importing eegducks

The next order of task is to specify the channel names and the EEG files. EEG datasets like ones found in *OpenNeuro* specify the names of the specific EEG channel locations in a separate *.tsv* file or *.locs* file. Thus, the location of this particular file needs to be stored in a variable. Next, the EEG dataset, which can be stored in different formats like *.fiff* or *.set*, among several others, needs to be specified along with the location variable stored. To load the file, we use the *load()* function provided by *eegducks*. Here's an example of the same -

```
1 # Specify the location of the channel location file
2 chan = "/content/sample/eeglab_chan32.locs"
3
4 # Save the EEG loaded in a a variable called 'raw' and specify the location file
   using 'locinf'
5 raw = eds.load("/content/sample/eeglab_data.set", locinf=chan)
```

Load() function example

The *Load()* function has the following parameters that can be modified if needed -

```
1 load(filepath, l_freq=1, h_freq=30, preload=True, ref="average", plot=True, locinf
   =None)
```

Load() function parameters

The output of the *load()* function includes the importing of the EEG data, applying a low and high pass filter, a notch filter, and ensuring the channel locations fit the EEG dataset.

2.2 Plotting using eegducks

Plotting in *eegducks* is a simple process. The following lines of code can be used to generate the plot seen in Figure 3 -

```
1 # 'eds' specifies the use of eegducks as when imported and '.plot' specifies the
   particular function from the library that needs to be used, and the dataset
   that was saved in 'raw' is specified
2
3 eds.plot(raw)
```

Plotting Continuous EEG data

The *plot()* function has the following parameters that can be modified if needed -

```
1 plot(raw, dur=5, n_chan=20)
```

plot() function parameters

Similarly, the power spectrum, as seen in Figure 4, can be plotted for the dataset -

```
1 #Power Spectrum Plot
2 eds.psdplt(raw)
3
4 # Plots the average
5 eds.psdplt(raw, avg=True)
```

Plotting the Power Spectrum

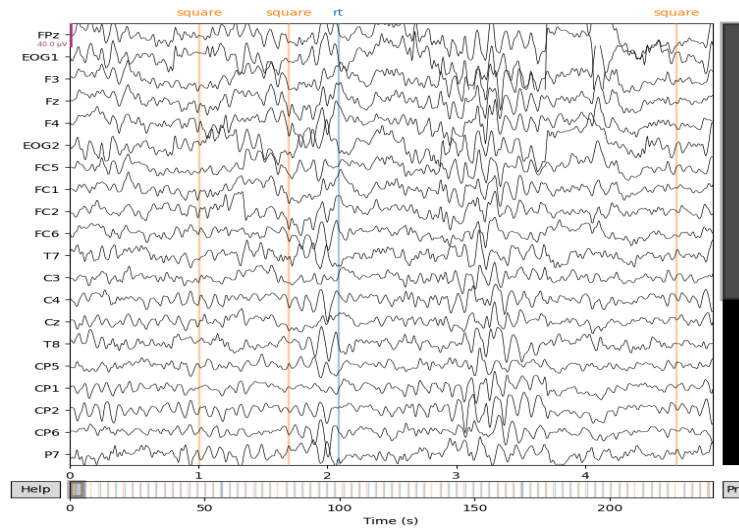


Figure 3: Continuous EEG data over time

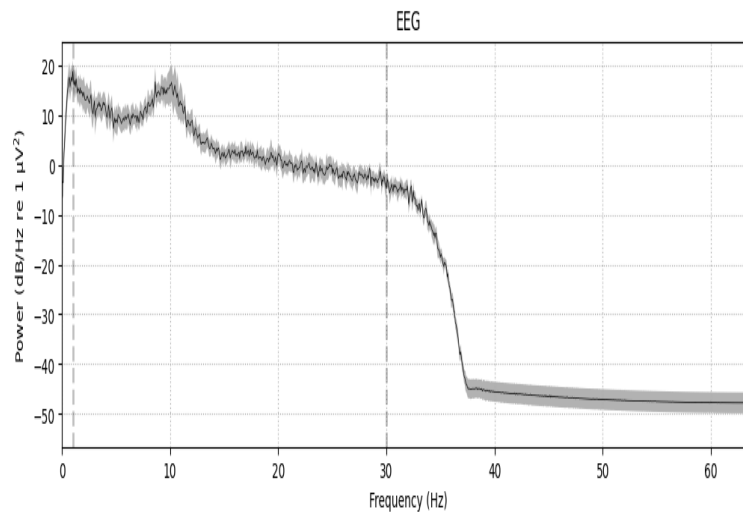


Figure 4: The Power Spectrum Plot

2.3 Specifying Epochs & Events

Certain EEG datasets have their epochs in the annotations itself whereas others specify the same in another file. For the former, the following suffices -

```
1 eds.epochs(raw)
```

Epochs (annotations)

In cases where the same is stored in another file, the file can be referenced in the *epoch()* function itself after specifying the location of the file -

```
1 # Specify Events file for Epochs
2 ev = "/content/s3/ev.tsv"
3
4 eds.epochs(raw, events_tsv=ev)
```

Epochs (Separate file)

The *epochs()* function also generates the plots, as seen in Figure 5 and Figure 6 . The *epochs()* function has the following parameters that can be modified if needed -

```
1 epochs(raw, event_id=2, tmin=-1, tmax=2, baseline=(-0.5, 0), reject=None, plot=
   True, events_tsv=None)
```

epoch() function parameters

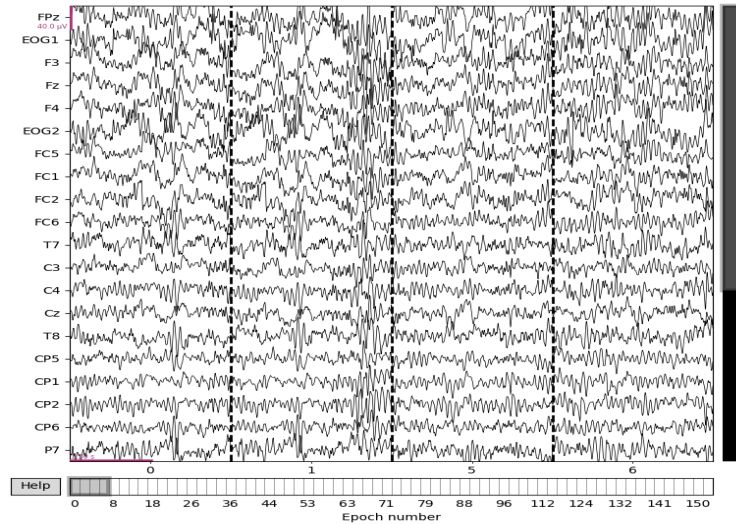


Figure 5: Epoch() function plot - a

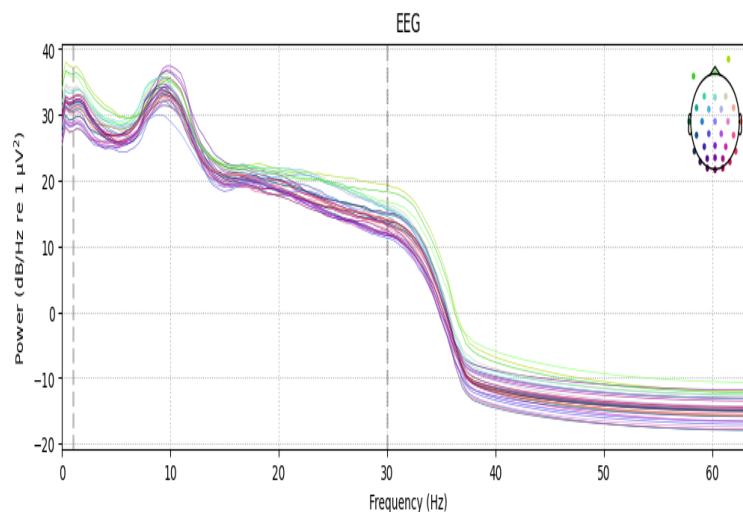


Figure 6: Epoch() function plot - b

2.4 Independent Component Analysis (ICA) using eegducks

Independent Component Analysis (ICA) is a crucial part of the EEG process due to its role in dealing with artifacts. To run an ICA using eegducks, the following code can be run -

```
1 # By default the number of components is set to 15, refer to the parameters below
   to learn how to change the same
2
3 eds.ica(raw)
```

ICA

The `ica()` function also generates the plot seen in Figure 7

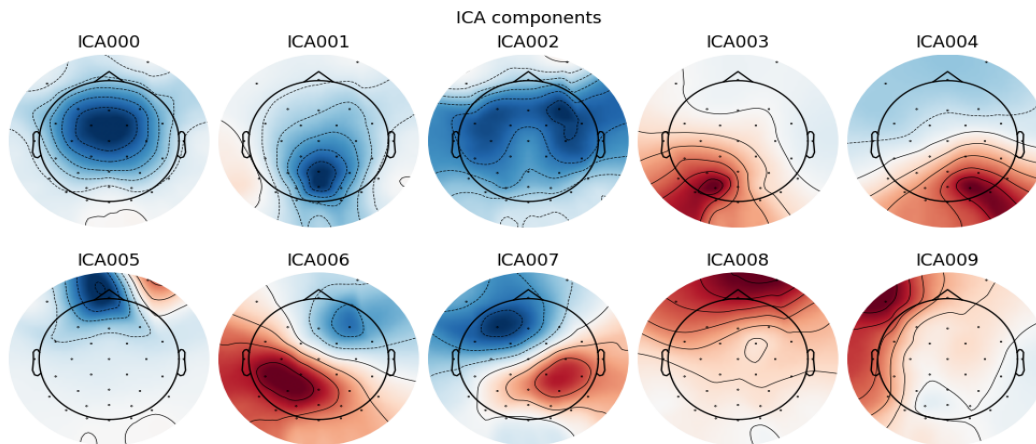


Figure 7: `ica()` function plot

The `ica()` function has the following parameters that can be modified if needed -

```
1 ica(raw, n_components=15, montage=None)
```

`ica()` function parameters

2.5 Plot Bands

To plot the different power bands, as seen in Figure 8, the function `pltbands()` can be run -

```
1 eds.pltbands(raw)
2
3 # or - pltbands(epochs, bands=None)
```

Plot Power Bands

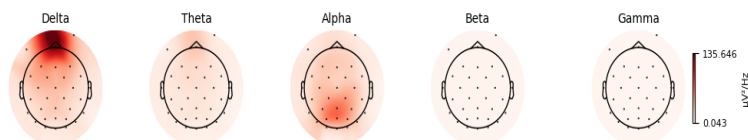


Figure 8: `pltbands()` function plot

3 Conclusion

The current paper introduces a lightweight and easy to use python library called 'eegducks', which serves as an introductory tool for students with little to no background in neuroimaging or python.

References

- [1] Alexandre Gramfort, Martin Luessi, Eric Larson, Denis A. Engemann, Daniel Strohmeier, Christian Brodbeck, Roman Goj, Mainak Jas, Teon Brooks, Lauri Parkkonen, and Matti S. Hämäläinen. MEG and EEG data analysis with MNE-Python. *Frontiers in Neuroscience*, 7(267):1–13, 2013.
- [2] Arnaud Delorme and Scott Makeig. Eeglab: an open source toolbox for analysis of single-trial eeg dynamics including independent component analysis. *Journal of neuroscience methods*, 134(1):9–21, 2004.